



Hacettepe Üniversitesi İstatistik Bölümü İST281 Bilgisayar Programlama Dersi

- **Liste Üreteçleri (List Comprehensions):**

Liste üreteçleri, liste üretmek için kısa yol sağlar (kısa kod). Amaç tekrarlanabilen işlemler ile gerekirse koşul ekleyerek yeni listeler oluşturmaktır.

```
kare=list()  
for x in range(1,7):  
    kare.append(x**2)  
print(kare)
```

[1, 4, 9, 16, 25, 36]

yerine,

```
kare=[x**2 for x in range(1,7)]
```

yazılabilir.

- **Örnek:** Liste değerlerinin iki katı ile yeni liste oluştur.

```
vektör = [-4, -2, 0, 2, 4]  
iki_kat=[x*2 for x in vektör]  
print(vektör)  
print(iki_kat)
```

```
[-4, -2, 0, 2, 4]  
[-8, -4, 0, 4, 8]
```

- **Örnek:** Listedeki negatif sayıları çıkartıp yeni liste oluştur.

```
vektör = [-4, -2, 0, 2, 4]  
pozitif=[x for x in vektör if x>=0]  
print(pozitif)
```

```
[0, 2, 4]
```

- **Örnek:** Listenin öğelerinde fonksiyon çağırılabilir.

```
vektör = [-4, -2, 0, 2, 4]
```

```
mutlak=[abs(x) for x in vektör]
```

```
print(mutlak)
```

```
[4, 2, 0, 2, 4]
```

- **Örnek:** Liste öğelerinde "method" çağırılabilir.

```
tazemeyve = ['muz', 'kızılcık', 'çarkıfelek']
```

```
t=[silah.capitalize() for silah in tazemeyve]
```

```
print(t)
```

```
['Muz', 'Kızılcık', 'Çarkıfelek']
```

- **Örnek:** Alt liste oluşturulabilir.

```
matris=[[x, x**2] for x in range(6)]  
print(matris)
```

```
[[0, 0], [1, 1], [2, 4], [3, 9], [4, 16], [5, 25]]
```

- **Örnek:** Alt liste öğeleri parantez içinde olmalıdır.

```
matris=[x, x**2 for x in range(6)]  
print(matris)
```

```
for deyiminde hata oluşur:  
SyntaxError: invalid syntax
```

- **Örnek:** İki tane for deyimi ile alt listeler, düz listeye dönüştürülebilir.

```
vektör = [[1,2,3], [4,5,6], [7,8,9]]
```

```
liste=[sayı for öge in vektör for sayı in öge]
```

```
print(liste)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- **İç içe Liste Üreteçleri (Nested List Comprehensions):**

3x4 matrisin liste gösterimi aşağıdaki gibi olsun.

```
matris = [  
    [1, 2, 3, 4],  
    [5, 6, 7, 8],  
    [9, 10, 11, 12],  
]
```

Liste üretici ile matrisin devriği:

```
devrik=[[satır[i] for satır in matris] for i in range(4)]  
print(devrik)
```

```
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

İç içe liste üretici ile eşdeğeri:

```
devrik = []  
for i in range(4):  
    devrik.append([satır[i] for satır in matris])  
print(devrik)
```

```
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

Liste üretici kullanmadan:

```
for i in range(4):  
    devrik_satır = []  
    for satır in matris:  
        devrik_satır.append(satır[i])  
    devrik.append(devrik_satır)  
print(devrik)
```


range() fonksiyonu, * işareti ve list() fonksiyonunu kullanmak:

`print(range(10))` `range(0, 10)`

`print(*range(10))` `0 1 2 3 4 5 6 7 8 9`

`liste=[*range(10)]`

`print(liste)` `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

```
liste=[[*range(5)],[*range(5)]]
```

```
print(liste)                [[0, 1, 2, 3, 4], [0, 1, 2, 3, 4]]
```

```
liste=[[*range(3)] for i in range(5)]
```

```
print(liste)                [[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
```

```
liste=list([*range(3)] for i in range(5))
```

```
print(liste)                [[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
```

```
liste=[range(10)]
```

```
print(liste)           [range(0, 10)]
```

```
liste=list(range(10))
```

```
print(liste)           [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
liste=list(range(-10, -100, -30))
```

```
print(liste)           [-10, -40, -70]
```

Yinelenebilir, gezilebilir nesneler (iterators):

Python'da iki tür yinelenebilir nesne vardır:

Ardışıklar (sequence): Bellekte kalıcı olarak yer alırlar. İndislerini kullanmak yolu ile öğelerine erişilebilir. Sonraki (next) işlemi yapılabilir. Şimdiye kadar incelenen yinelenebilir nesneler bu türdedir. Örneğin metin (string), liste (list) gibi.

Üreteçler (generator): Bellekte kalıcı yer kaplamazlar. Sırası geldikçe üretilirler ve kullanıldıktan sonra bellekten silinirler. Liste üreteçlerinde kullandığımız kısa kodlar bu türdendir. İndisleri yoktur.

Örnek:

```
g = (x*x for x in range(5))
```

```
print(type(g))
```

```
<class 'generator'>
```

Örnek: Üretecini bir kez kullanılabilmesi, sonrasında bellekten silinmesi.

```
g = (x*x for x in range(5))
```

```
for k in g:
```

```
    print(k, end=" ")
```

0 1 4 9 16

Üretilen g üretecini ikinci kez yazmak isterseniz:

```
for k in g:
```

```
    print(k, end=" ")
```

Çıktı vermez. Fakat boş olarak bellekte bulunur.

```
print(type(g))          <class 'generator'>
```

yield deyimi:

Bir fonksiyonu üretece (generator) dönüştürür. Fonksiyon çağırıldığında, çalışıp sonlanmaz, üretece dönüşür. Genellikle bir döngü ile birçok kez çağırılır. İlk çağırıldığında, yield deyimine kadar çalışır, ürettiği değeri, return deyimi gibi döndürür. tekrar çağırıldığında kaldığı yerden, yani yield deyiminin sonrasında çalışmaya devam eder.

Örnek: Bir listenin kareleri.

```
def kare(lst):  
    for k in lst:  
        yield k*k
```

Kullanılması:

```
liste=[5,6,7]  
for i in kare(liste):  
    print(i)
```

25

36

49

Örnek: Sonsuz döngü ile karelerin üretilmesi.

```
def SonrakiKare():
```

```
    i = 1
```

```
    while True:
```

```
        yield i*i
```

```
        i += 1    # Sonraki çağırımda buradan devam eder.
```

Kullanılması:

```
for sayı in SonrakiKare():
```

```
    if sayı > 100:
```

```
        break
```

```
print(sayı, end=" ")
```

1 4 9 16 25 36 49 64 81 100

enumerate() fonksiyonu: Bir numaralandırma nesnesi döndürür.

enumerate(<yinelenenebilir nesne>, start=0)

yinelenenebilir nesne: Ardışık bir dizi veya yinelemeyi destekleyen bir nesne olmalıdır.

start=0: Varsayılan değeri sıfırdır, yazılmayabilir. Numaralandırmaya başlanacak değeri gösterir.

Örnek:

```
mevsimler = ['İlk Bahar', 'Yaz', 'Son Bahar', 'Kış']
```

```
liste=list(enumerate(mevsimler))
```

```
[(0, 'İlk Bahar'), (1, 'Yaz'), (2, 'Son Bahar'), (3, 'Kış')]
```

Enumerate fonksiyonunun döndürdüğü sayaç değerleri demet (tuple) biçimindedir.

Örnek:

```
mevsimler = ['İlk Bahar', 'Yaz', 'Son Bahar', 'Kış']
```

```
for sayaç, değer in enumerate(mevsimler):
```

```
    print(sayaç, değer)
```

```
0 İlk Bahar  
1 Yaz  
2 Son Bahar  
3 Kış
```

Sorular

- **Soru 1:** Aşağıdaki liste üretici (kısa kod) ile yazılmış program parçasının açık halini yazınız.

```
sayi_listesi = [ x for x in range(20) if x % 2 == 0]  
print(sayi_listesi)
```

- **Soru 2:** Aşağıdaki liste üretici (kısa kod) ile yazılmış program parçasının açık halini yazınız.

```
sayi_listesi = [y for y in range(100) if y % 2 == 0 if y % 5 == 0]  
print(sayi_listesi)
```

- **Soru 3:** Aşağıda verilen program parçasının eşdeğerini liste üretici yöntemi ile yazınız.

```
liste=[]
for i in range(10):
    if i%2==0:
        liste.append("Çift")
    else:
        liste.append("Tek")
print(liste)
```

- **Soru 4:** Liste formunda olan 2x4 boyutlu matrisin satırlarında bulunan öğeleri 2'şerli ayırıp, 4x2 boyutlu yeni bir matris oluşturun. Çözümünüzü açık ve liste üretici formlarında gösterin.

2x4 boyutlu matris: $[[1, 2, 3, 4], [5, 6, 7, 8]]$ ise yeni matris:
 $[[1, 2], [3, 4], [5, 6], [7, 8]]$ biçiminde olsun.

Ek çalışma: Aynı işlemi, $m \geq 1$, $n \geq 4$ ve n değeri 2'nin katı olmak üzere $m \times n$ boyut için genelleştirin. Veri girişinde m ve n değerlerinin uygun olup olmadığını denetleyin. Matrisi okuyun.

- **Soru 5:** Öğe sayısı kullanıcıdan istenen bir listeyi okuyun. Çözümünüzü önce açık döngü ile sonra kısa kodlama (liste üretici) ile yapın.
- **Soru 6:** Satır ve sütun sayıları ekrandan istenen bir matrisi, liste yapısı formunda okuyun. Çözümünüz üç farklı biçimde olsun.
 1. Açık formda içi içe iki döngü ile.
 2. İç içe liste üretici ile, bir açık döngü ve bir liste üretici ile.
 3. Tek satırda bir liste üretici ile.